

LLM Learning Plan & Reading List

Resources, Courses, and Instrumental Papers

Bernard

May 24, 2026

How to Use This Plan

This plan is sequenced so that each step builds on the previous. The first three steps build theoretical foundations; the middle steps bridge to engineering; the final steps reach the research frontier. Within each step, the first resource is the primary recommendation; the second is a complementary alternative or deeper dive.

1 Learning Plan

- Step 1: Information Theory** *foundations — 2–4 weeks*
1. Cover & Thomas, [Elements of Information Theory](#), 2nd ed., Wiley 2006. *Ch.1–4, 8–10 (skippable: Ch.5–7)*
 2. MacKay, [Information Theory, Inference, and Learning Algorithms](#), CUP 2003. *Free online*
- Unique: marries info theory to ML/Bayesian inference in one unified narrative.
- Step 2: Deep Learning Mathematics** *foundations — 4–6 weeks*
1. Goodfellow et al., [Deep Learning](#), MIT Press 2016. *Free online*
Ch.1–6 (math + ML basics), Ch.9–10 (CNNs, RNNs/attention).
 2. Chen & Siegel, [Mathematical Foundations of Deep Learning Models and Algorithms](#), AMS 2025.
Rigorous proofs of UAT, NTK, mean-field, backward propagation. Ch.13 covers transformers.
- Step 3: LLM Architecture & Theory** *core theory — 3–5 weeks*
1. Vaswani et al., [Attention Is All You Need](#), NeurIPS 2017. *Primary source*
Read alongside [Large Language Models: A Mathematical Formulation](#) (2026) for a modern mathematical treatment covering tokenisation, training, RLHF.
 2. [Support Tokens, Stability Margins, and a New Foundation for Robust LLMs](#) (2026).
Re-frames causal self-attention as a large-margin classifier; introduces the “support tokens” concept and a Bayesian MAP penalty.
- Step 4: GPU Architecture & CUDA** *hardware — 4–6 weeks*
1. [CUDA C++ Programming Guide](#) (NVIDIA official). *Reference*
Supplement with [siboehm, CUDA by Examples](#) (blog + code).

2. GPU Performance Engineering Resources, wafer-ai. *Curated list*
The definitive curated guide: roofline model, shared memory, tensor cores, flash attention, PagedAttention, CUDA graphs, warp specialisation.

Step 5: LLM Inference Optimisation *engineering — 3–4 weeks*

1. [CUDA Optimization Techniques for Local LLMs: The Complete 2026 Guide](#).

Practical ranking of optimisations by speedup/effort/risk: FlashAttention, KV-cache quantisation, FP8, speculative decoding, CUDA graphs.

2. [Accelerating Large Language Model Decoding with Speculative Decoding](#) (Chen et al., 2023) + [FlashAttention](#) (Dao et al., 2022).

Read both end-to-end; then study the KV cache in the [PagedAttention](#) paper (Kwon et al., 2023).

Step 6: Production Serving *deployment — 2–3 weeks*

1. [vLLM documentation](#) + [GitHub](#).

Set up a local server; understand continuous batching, prefix caching, tensor parallelism, AWQ quantisation.

2. [TensorRT-LLM](#) (NVIDIA) + [llama.cpp](#).

Compare the two: TRT-LLM for max throughput on datacenter GPUs, llama.cpp for edge/CPU inference. Benchmark both on the same model.

Step 7: Software Engineering for LLMs *application — 2–3 weeks*

1. Lewis et al., [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#), NeurIPS 2020.

The foundational RAG paper. Then build a RAG pipeline with [LlamaIndex](#).

Supplement: Wei et al., [Chain-of-Thought Prompting Elicits Reasoning](#), NeurIPS 2022.

2. Implement an LLM agent with [LangChain](#) that uses tools (calculator, search, database). Study: Ouyang et al., [Training Language Models to Follow Instructions with Human Feedback](#), NeurIPS 2022.

Step 8: Research Frontier: Long Context, Sparsity, RLVR *cutting edge — 4–6 weeks*

1. [Sparse Feature Attention: Dimension-Level Sparsity for Efficient Long-Context LLMs](#) (2026).

Proposes learning k -sparse query/key codes; introduces FlashSFA. Matches dense perplexity at $2.5\times$ speedup.

2. [LongRLVR: Reinforcement Learning with Verifiable Rewards for Long-Context Reasoning](#) (2026).

Proves outcome-only RLVR suffers vanishing gradients for contextual grounding; introduces a dense context reward. Improves RULER from 17 $\rightarrow$ 88.

Step 9: Depth Scaling, Representation Geometry *emerging theory — 3–4 weeks*

1. [Why Gradual Depth Growth Improves Reasoning in LLMs](#) (2026, ICLR).

Mechanistic analysis: depth-grown models use late layers; standard models suffer “Curse of Depth” (late layers are redundant). LIDAS algorithm.

2. [Understanding Embedding Condensation in LLMs](#) (2026).

Smaller models exhibit collapsed embedding geometry; proposes dispersion loss that recovers the representational diversity of larger models.

2 Instrumental Papers

Foundations

- (1) **Attention Is All You Need.** Vaswani et al. NeurIPS 2017. arxiv.org/abs/1706.03762
The paper that started everything. Transformer architecture, multi-head attention, positional encodings.
- (2) **Scaling Laws for Neural Language Models.** Kaplan et al. 2020. arxiv.org/abs/2001.08361
Power-law scaling of loss with parameters, data, and compute. Established the empirical foundation.
- (3) **Training Compute-Optimal Large Language Models.** Hoffmann et al. (Chinchilla). NeurIPS 2022. arxiv.org/abs/2203.15556
Models and data should be scaled equally. Refuted the Kaplan scaling recommendations.
- (4) **A Mathematical Theory of Communication.** Shannon. Bell System Technical Journal 1948. [Free PDF](#)
The intellectual foundation of all information-theoretic reasoning about LLMs.
- (5) **The Information Bottleneck Method.** Tishby, Pereira, Bialek. Allerton 1999. arxiv.org/abs/physics/9906037
Framework for understanding hidden representations as compressing input while preserving task signal.

Architecture and Training

- (6) **Language Models are Few-Shot Learners (GPT-3).** Brown et al. NeurIPS 2020. arxiv.org/abs/2005.14165
Demonstrated in-context learning at scale; the paradigm shift from fine-tuning to prompting.
- (7) **Training Language Models to Follow Instructions with Human Feedback (InstructGPT).** Ouyang et al. NeurIPS 2022. arxiv.org/abs/2203.02155
The canonical RLHF paper: PPO + KL penalty for alignment.
- (8) **Llama 2: Open Foundation and Fine-Tuned Chat Models.** Touvron et al. 2023. arxiv.org/abs/2307.09288
GQA, SwiGLU, RoPE, RLHF recipe. The reference open-source architecture.
- (9) **The Llama 3 Herd of Models.** Dubey et al. 2024. arxiv.org/abs/2407.21783
Scaling to 405B; instruction tuning, tool use, multilingual, long-context.
- (10) **DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.** Guo et al. 2025. arxiv.org/abs/2501.12948
RLVR with verifiable rewards produces emergent reasoning (the “aha moment”).

Inference and Systems

- (11) **FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.** Dao et al. NeurIPS 2022. arxiv.org/abs/2205.14135
Tiling + recomputation eliminates $O(N^2)$ memory. Essential reading for GPU optimisation.
- (12) **Efficient Memory Management for Large Language Model Serving with PagedAttention.** Kwon et al. (vLLM). SOSP 2023. arxiv.org/abs/2309.06180
Virtual memory for KV cache; eliminates fragmentation. Continuous batching.
- (13) **Fast Inference from Transformers via Speculative Decoding.** Leviathan et al. ICML 2023. arxiv.org/abs/2211.17192
Draft-verify paradigm; expected speedup analysis.
- (14) **GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers.** Frantar et al. ICLR 2023. arxiv.org/abs/2210.17323
Optimal Brain Quantiser adapted for LLMs; the standard PTQ baseline.

- (15) **AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration.** Lin et al. MLSys 2024. arxiv.org/abs/2306.00978
Observes 1% of weights (salient channels) are disproportionately important; protects them during quantisation.

Engineering and Methods

- (16) **Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.** Wei et al. NeurIPS 2022. arxiv.org/abs/2201.11903
CoT increases effective depth; enables problems unsolvable by direct prompting.
- (17) **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.** Lewis et al. NeurIPS 2020. arxiv.org/abs/2005.11401
The RAG paradigm: index, retrieve, generate.
- (18) **Tree of Thoughts: Deliberate Problem Solving with Large Language Models.** Yao et al. NeurIPS 2023. arxiv.org/abs/2305.10601
Search over reasoning trees; connects LLM decoding to classical AI search.
- (19) **HarmonyDream: No Need for Task-Specific Decoders in World Models.** Robine et al. ICLR 2025. arxiv.org/abs/2502.12872
Architectural innovation for latent-state world models; demonstrates shared representations.
- (20) **React: Synergizing Reasoning and Acting in Language Models.** Yao et al. ICLR 2023. arxiv.org/abs/2210.03629
Interleaved reasoning traces and tool actions; the foundation of modern agent frameworks.

Research Frontier (2025–2026)

- (21) **Large Language Models: A Mathematical Formulation.** Brennan et al. 2026. arxiv.org/abs/2601.22170
Rigorous mathematical framework covering tokenisation, architecture, training, and deployment.
- (22) **Support Tokens, Stability Margins, and a New Foundation for Robust LLMs.** 2026. arxiv.org/abs/2602.22271
Causal self-attention induces a margin-to-degeneracy barrier; support tokens emerge analogous to SVMs.
- (23) **Language Models are Almost Surely Injective.** 2026. arxiv.org/abs/2510.15511
Proves decoder-only transformers are injective; SIPIT algorithm recovers input from hidden states.
- (24) **Sparse Feature Attention.** 2026. arxiv.org/abs/2603.22300
Dimension-level sparsity for attention; FlashSFA kernel. k -sparse queries/keys. $2.5\times$ speedup.
- (25) **LongRLVR: RL with Verifiable Rewards for Long-Context Reasoning.** 2026. arxiv.org/abs/2603.02146
Proves outcome-only RLVR has vanishing gradients for contextual grounding. Dense context reward.
- (26) **Why Gradual Depth Growth Improves Reasoning in LLMs.** 2026 (ICLR). [Open-Review](#)
Depth-grown models escape the Curse of Depth; late layers remain useful. LIDAS algorithm.
- (27) **Understanding Embedding Condensation in LLMs.** 2026. arxiv.org/abs/2602.00217
Smaller models collapse embedding diversity; dispersion loss recovers large-model geometry.
- (28) **In-Context Policy Optimization.** 2026 (ICLR). arxiv.org/abs/2603.01335
Theoretical framework proving one-layer linear self-attention can execute policy optimisa-

tion in-context.

Last updated: May 24, 2026